PRESENTACIÓN DEL DISEÑO DE PRUEBAS A APLICAR AL PROYECTO

Sistema de Reserva de Hotel · Aseguramiento de la Calidad de Software (QA)

Docente: Gerber Alexis Gómez Hidalgo | Fecha de entrega: 22 de octubre de 2026

Integrantes: Ariel Isai Ascencio Ramos · Joselin Esmeralda Herrera Flores · Carlos Alonso Miranda Rivas · Camila Antonia Calderón Cortez

Índice

1. Introducción

2. Objetivo

3. Capturas de los casos programados

3.1. Ariel Isai Ascencio Ramos — US-16 (#1040)

3.2. Joselin Esmeralda Herrera Flores — US-15 (#1037)

3.3. Carlos Alonso Miranda Rivas — US-14 (#1034)

3.4. Camila Antonia Calderón Cortez — US-SEC-01 (#1043)

3.5. Resultado de la ejecución de las pruebas

4. Evidencias y capturas de casos en Azure DevOps

4.1. Plan Maestro de Pruebas (Test Plan Maestro #1054)

4.2. Detalle de casos de prueba registrados (TC-01 a TC-08)

4.3. Trazabilidad con requerimientos en Azure Boards (PBIs y Kanban)

4.4. Reporte de progreso y métricas de calidad (Progress Report)

4.5. Matriz de casos de prueba registrados en Azure Test Plans

5. Conclusiones y reflexión por integrante

6. Enlace del repositorio en GitHub

1. Introducción

El Sistema de Reserva de Hotel es una aplicación desarrollada con ASP.NET Core MVC sobre .NET, con acceso a datos mediante Entity Framework Core, que permite gestionar habitaciones, reservas, tarifas por temporada, reportes y control de acceso por roles. Por tratarse de un sistema transaccional, un error en el cálculo de una tarifa, en la validación de fechas o en la detección de habitaciones ocupadas se traduce directamente en pérdidas económicas y en una mala experiencia para el huésped; por ello la calidad del software debe verificarse de forma planificada y rigurosa en cada nivel de la arquitectura.

Para garantizar un aseguramiento de la calidad (QA) integral bajo la norma ISO/IEC/IEEE 29119, el equipo organizó el trabajo a partir de las historias de usuario (US-14, US-15, US-16 y US-SEC-01) registradas en Azure DevOps y distribuyó entre los cuatro integrantes el diseño y la programación de las pruebas en tres niveles fundamentales:

- Pruebas Unitarias (Moq / AAA): verificación aislada de la lógica y reglas de negocio de los servicios (UT-01 a UT-21).
- Pruebas de Aceptación (UAT): validación de flujos de usuario y reglas críticas del sistema (TC-01 a TC-08) vinculadas al Test Plan #1054 de Azure DevOps.
- Pruebas de API (HTTP / REST): validación de contratos, códigos de estado HTTP y persistencia en endpoints (API-01 a API-16).

Este documento presenta el resultado consolidado de ese diseño: primero las capturas del código de los casos programados por cada integrante (clasificados en pruebas unitarias, de aceptación y de API), luego las evidencias de los casos documentados en Azure DevOps, y finalmente las conclusiones y la reflexión individual. Como cierre se incluye el enlace del repositorio oficial en GitHub.

2. Objetivo

Objetivo general. Elaborar el diseño de los casos de prueba a aplicar al Sistema de Reserva de Hotel, estructurados en pruebas unitarias, pruebas de aceptación y pruebas de API, garantizando que las reglas de negocio críticas queden verificadas de manera automatizada y con trazabilidad formal en Azure DevOps.

Objetivos específicos:

- Diseñar y programar pruebas unitarias aisladas con xUnit y Moq para los servicios de negocio (UT-01 a UT-21).
- Diseñar y programar pruebas de aceptación automatizadas (TC-01 a TC-08) vinculadas al Test Plan #1054 de Azure DevOps.
- Diseñar y programar pruebas de API sobre los endpoints HTTP REST con WebApplicationFactory y base de datos en memoria (API-01 a API-16).
- Distribuir las responsabilidades de QA entre los cuatro integrantes conforme a las historias de usuario asignadas.
- Consolidar en un documento formal las evidencias de los casos programados y documentados en Azure DevOps.

3. Capturas de los casos programados

Las siguientes capturas corresponden al código real de las pruebas almacenadas en el repositorio, organizadas por integrante. Cada integrante diseñó y programó los tres niveles de prueba que le fueron asignados: Pruebas Unitarias, Pruebas de Aceptación y Pruebas de API.

3.1. Ariel Isai Ascencio Ramos — US-16 (#1040): Reglas Cronológicas y Autenticación

Cobertura asignada: Pruebas Unitarias (UT-01 a UT-06), Pruebas de Aceptación (TC-01 y TC-07) y Pruebas de API (API-01 a API-05).

ValidadorFechasTests.cs — Pruebas Unitarias (UT-01/UT-03) — Ariel Isai Ascencio Ramos

```
 9  [Fact]
10  public void FechasValidas_EntradaFuturaYMenorQueSalida_RetornaTrue()
11  {
12      // Arrange
13      var entrada = DateTime.Today.AddDays(2);
14      var salida = DateTime.Today.AddDays(5);
15
16      // Act
17      var resultado = ValidadorFechas.FechasValidas(entrada, salida);
18
19      // Assert
20      Assert.True(resultado);
21  }
22
23  [Fact]
24  public void FechasValidas_EntradaHoyYSalidaManana_RetornaTrue()
25  {
26      // Arrange
27      var entrada = DateTime.Today;
28      var salida = DateTime.Today.AddDays(1);
29
30      // Act
31      var resultado = ValidadorFechas.FechasValidas(entrada, salida);
32
33      // Assert
34      Assert.True(resultado);
35  }
36
37  [Fact]
38  public void FechasValidas_EntradaMayorQueSalida_RetornaFalse()
39  {
40      // Arrange
41      var entrada = DateTime.Today.AddDays(5);
42      var salida = DateTime.Today.AddDays(2);
43
44      // Act
45      var resultado = ValidadorFechas.FechasValidas(entrada, salida);
46
47      // Assert
48      Assert.False(resultado);
49  }
```

Figura 1. Validación unitaria de fechas: rango válido vs entrada mayor o igual a salida.

UnitTests_TC05_TC06.cs — Prueba de Aceptación (TC-01) — Ariel Isai Ascencio Ramos

```
185  [Fact]
186  public async Task TC_01_Autenticacion_LoginAsync_AdministradorAutorizado()
187  {
188      using var context = _fixture.CreateContext();
189      var uow = new UnitOfWork(context);
190      var authService = new AuthService(uow);
191
192      // 1. Login exitoso con credenciales de administrador
193      var loginDto = new LoginDto
194      {
195          Email = "admin@facturacion.com",
196          Password = "Admin123!"
197      };
198      var (success, message, usuario) = await authService.LoginAsync(loginDto);
199
200      Assert.True(success, "El inicio de sesión debió ser exitoso.");
201      Assert.NotNull(usuario);
202      Assert.Equal("admin@facturacion.com", usuario.Email);
203      Assert.Contains("Administrador", usuario.Roles);
204
205      // 2. Intento fallido con contraseña incorrecta
206      var (failSuccess, failMessage, failUser) = await authService.LoginAsync(new LoginDto
207      {
208          Email = "admin@facturacion.com",
209          Password = "PasswordInvalida999!"
210      });
211
212      Assert.False(failSuccess, "El login con credenciales inválidas debe fallar.");
213      Assert.Null(failUser);
214  }
```

Figura 2. Caso de aceptación TC-01: autenticación de administrador con AuthService.LoginAsync.

UnitTests_TC05_TC06.cs — Prueba de Aceptación (TC-07) — Ariel Isai Ascencio Ramos

```
369  [Fact]
370  public async Task TC_07_ValidarPasswordAsync_VerificacionHashSHA256()
371  {
372      using var context = _fixture.CreateContext();
373      var uow = new UnitOfWork(context);
374      var authService = new AuthService(uow);
375
376      // Escenario 1: Credenciales válidas del administrador sembrado
377      bool passValida = await authService.ValidarPasswordAsync("admin@facturacion.com", "Admin123!");
378      Assert.True(passValida, "La contraseña válida 'Admin123!' debió ser validada como true por
ValidarPasswordAsync.");
379
380      // Escenario 2: Contraseña errónea con correo existente
381      bool passErronea = await authService.ValidarPasswordAsync("admin@facturacion.com", "ClaveIncorrecta999!");
382      Assert.False(passErronea, "Una contraseña inválida debió retornar false.");
383
384      // Escenario 3: Correo inexistente en el sistema
385      bool passInexistente = await authService.ValidarPasswordAsync("noexiste@hotel.com", "Admin123!");
386      Assert.False(passInexistente, "Un usuario inexistente debió retornar false.");
387  }
```

Figura 3. Caso de aceptación TC-07: validación criptográfica de contraseña contra el hash SHA-256.

ValidadorFechasIntegrationTests.cs — Pruebas de API (API-02/API-03) — Ariel Isai Ascencio Ramos

```
60  [Fact]
61  public async Task PostReserva_ConFechaEntradaMayorQueSalida_Retorna400BadRequest()
62  {
63      // Arrange (US-16 #1040 - Caso Rango Invertido)
64      var reservaInvertida = new Reserva
65      {
66          HabitacionId = 1,
67          NombreCliente = "Cliente Invertido",
68          FechaEntrada = DateTime.Today.AddDays(25),
69          FechaSalida = DateTime.Today.AddDays(20),
70          PrecioTotal = 150m
71      };
72
73      // Act
74      var response = await _client.PostAsJsonAsync("/Reserva/Crear", reservaInvertida);
75
76      // Assert
77      response.StatusCode.Should().Be(HttpStatusCode.BadRequest);
78  }
79
80  [Fact]
81  public async Task PostReserva_ConFechaEntradaIgualASalida_Retorna400BadRequest()
82  {
83      // Arrange (US-16 #1040 - Caso Estancia Cero Noches)
84      var reservaMismoDia = new Reserva
85      {
86          HabitacionId = 1,
87          NombreCliente = "Cliente Mismo Dia",
88          FechaEntrada = DateTime.Today.AddDays(5),
89          FechaSalida = DateTime.Today.AddDays(5),
90          PrecioTotal = 100m
91      };
92
93      // Act
94      var response = await _client.PostAsJsonAsync("/Reserva/Crear", reservaMismoDia);
95
96      // Assert
97      response.StatusCode.Should().Be(HttpStatusCode.BadRequest);
98  }
```

Figura 4. Pruebas de API HTTP: POST /Reserva/Crear rechazando fechas invertidas y estancia 0 noches (400 BadRequest).

3.2. Joselin Esmeralda Herrera Flores — US-15 (#1037): Disponibilidad y Catálogo

Cobertura asignada: Pruebas Unitarias (UT-07 a UT-12), Pruebas de Aceptación (TC-04 y TC-05) y Pruebas de API (API-06 a API-08).

DisponibilidadServiceTests.cs — Pruebas Unitarias (UT-09/UT-10) — Joselin Esmeralda Herrera Flores

```
74  [Fact]
75  public async Task ObtenerDisponiblesAsync_ConReservaActivaEnConflicto_ExcluyeHabitacionOcupada()
76  {
77      // Arrange
78      var habitaciones = new List<Habitacion>
79      {
80          new Habitacion { Id = 1, Numero = 101, Activa = true },
81          new Habitacion { Id = 2, Numero = 102, Activa = true }
82      };
83
84      var reservas = new List<Reserva>
85      {
86          new Reserva
87          {
88              Id = 1,
89              HabitacionId = 1,
90              FechaEntrada = DateTime.Today.AddDays(2),
91              FechaSalida = DateTime.Today.AddDays(5),
92              Estado = "Activa"
93          }
94      };
95
96      _habitacionRepoMock.Setup(r => r.ObtenerTodasAsync()).ReturnsAsync(habitaciones);
97      _reservaRepoMock.Setup(r => r.ObtenerTodasAsync()).ReturnsAsync(reservas);
98
99      var entrada = DateTime.Today.AddDays(3);
100     var salida = DateTime.Today.AddDays(6);
101
102     // Act
103     var disponibles = await _service.ObtenerDisponiblesAsync(entrada, salida);
104
105     // Assert
106     Assert.Single(disponibles);
107     Assert.Equal(2, disponibles[0].Id);
108 }
109
110 [Fact]
111 public async Task ObtenerDisponiblesAsync_ConReservaCanceladaEnMismoRango_NoExcluyeHabitacion()
112 {
113     // Arrange
114     var habitaciones = new List<Habitacion>
115     {
```

Figura 5. Pruebas unitarias de DisponibilidadService con Moq: exclusión de habitación ocupada por reserva activa.

UnitTests_TC05_TC06.cs — Prueba de Aceptación (TC-04) — Joselin Esmeralda Herrera Flores

```
264 [Fact]
265 public async Task TC_04_ReporteRangoFechas_InclusionesYExclusiones()
266 {
267     using var context = _fixture.CreateContext();
268 
269     // Asegurar que exista una reserva en noviembre y otra en diciembre
270     if (!context.Reservas.Any(r => r.NombreCliente == "Huesped_Diciembre_TC04"))
271     {
272         context.Reservas.Add(new Reserva
273         {
274             HabitacionId = 1,
275             NombreCliente = "Huesped_Diciembre_TC04",
276             FechaEntrada = new DateTime(2026, 12, 10),
277             FechaSalida = new DateTime(2026, 12, 15),
278             PrecioTotal = 600.00m,
279             Estado = "Activa"
280         });
281         await context.SaveChangesAsync();
282     }
283 
284     var inicio = new DateTime(2026, 11, 1);
285     var fin = new DateTime(2026, 11, 30);
286 
287     // Ejecución del filtro tal como lo hace ReportesController.Reservas(inicio, fin)
288     var query = context.Reservas
289         .Where(r => r.FechaEntrada >= inicio && r.FechaSalida <= fin)
290         .ToList();
291 
292     // Inclusión: reserva de noviembre presente
293     Assert.Contains(query, r => r.NombreCliente == "Cliente Existente QA");
294 
295     // Exclusión: reserva de diciembre excluida
296     Assert.DoesNotContain(query, r => r.NombreCliente == "Huesped_Diciembre_TC04");
297 }
```

Figura 6. Caso de aceptación TC-04: filtrado de reservas por rango de fechas en reporte gerencial.

UnitTests_TC05_TC06.cs — Prueba de Aceptación (TC-05) — Joselin Esmeralda Herrera Flores

```
299  [Fact]
300  public async Task TC_05_ValidarAnticolision_DisponibilidadService_Y_ReservaService()
301  {
302      using var context = _fixture.CreateContext();
303      var repo = new ReservaRepository(context);
304      var reservaService = new ReservaService(repo);
305
306      // Asegurar aislamiento: solo la reserva base del 10 al 15 en Habitación 1
307      var extras = context.Reservas.Where(r => r.HabitacionId == 1 && r.NombreCliente != "Cliente Existente
QA").ToList();
308      if (extras.Any())
309      {
310          context.Reservas.RemoveRange(extras);
311          await context.SaveChangesAsync();
312      }
313
314      // Caso 1: Solapamiento frontal (entrada 8 a salida 12 con reserva del 10 al 15) -> false
315      bool dispFrontal = await reservaService.ValidarDisponibilidadAsync(1, new DateTime(2026, 11, 8), new
DateTime(2026, 11, 12));
316      Assert.False(dispFrontal, "El solapamiento frontal debió retornar false.");
317
318      // Caso 2: Solapamiento trasero (entrada 13 a salida 17) -> false
319      bool dispTrasero = await reservaService.ValidarDisponibilidadAsync(1, new DateTime(2026, 11, 13), new
DateTime(2026, 11, 17));
320      Assert.False(dispTrasero, "El solapamiento trasero debió retornar false.");
321
322      // Caso 3: Solapamiento interno (entrada 11 a salida 14) -> false
323      bool dispInterno = await reservaService.ValidarDisponibilidadAsync(1, new DateTime(2026, 11, 11), new
DateTime(2026, 11, 14));
324      Assert.False(dispInterno, "El solapamiento interno debió retornar false.");
325
326      // Caso 4: Rango libre (entrada 25 a salida 28) -> true
327      bool dispLibre = await reservaService.ValidarDisponibilidadAsync(1, new DateTime(2026, 11, 25), new
DateTime(2026, 11, 28));
328      Assert.True(dispLibre, "El rango libre debió retornar true.");
329  }
```

Figura 7. Caso de aceptación TC-05: control anticolisión de disponibilidad ante fechas solapadas.

DisponibilidadIntegrationTests.cs — Prueba de API (API-07) — Joselin Esmeralda Herrera Flores

```
45  [Fact]
46  public async Task GetDisponibilidad_EnRangoConReservaActiva_ExcluyeHabitacionOcupada()
47  {
48      // Arrange (US-15 #1037 - Conflicto con la reserva semilla existente del día 10 al 15)
49      var entrada = DateTime.Today.AddDays(11).ToString("yyyy-MM-dd");
50      var salida = DateTime.Today.AddDays(14).ToString("yyyy-MM-dd");
51
52      // Act
53      var response = await _client.GetAsync($"/Disponibilidad?entrada={entrada}&salida={salida}");
54
55      // Assert
56      response.StatusCode.Should().Be(HttpStatusCode.OK);
57
58      var disponibles = await response.Content.ReadFromJsonAsync<List<Habitacion>>();
59      disponibles.Should().NotBeNull();
60      disponibles.Should().NotContain(h => h.Id == 1);
61  }
```

Figura 8. Prueba de API HTTP: GET /Disponibilidad excluyendo habitación ocupada con reserva activa.

3.3. Carlos Alonso Miranda Rivas — US-14 (#1034): Consultas y Lectura

Cobertura asignada: Pruebas Unitarias (UT-13 a UT-16), Pruebas de Aceptación (TC-02 y TC-08) y Pruebas de API (API-09 a API-12).

ReservaPublicaServiceTests.cs — Pruebas Unitarias (UT-13/UT-14) — Carlos Alonso Miranda Rivas

```
23  #region Carlos Alonso Miranda Rivas - US-14 (#1034) - Consultas y Lectura
24
25  [Fact]
26  public async Task ObtenerPorDia_FechaValida_RetornaReservas()
27  {
28      // Arrange
29      var fecha = DateTime.Today;
30      var reservasSimuladas = new List<Reserva>
31      {
32          new Reserva { Id = 1, NombreCliente = "Cliente 1", FechaEntrada = fecha, FechaSalida = fecha.AddDays(2)
},
33          new Reserva { Id = 2, NombreCliente = "Cliente 2", FechaEntrada = fecha, FechaSalida = fecha.AddDays(1) }
34      };
35
36      _reservaRepoMock.Setup(r => r.ObtenerPorDiaAsync(fecha)).ReturnsAsync(reservasSimuladas);
37
38      // Act
39      var resultado = await _service.ObtenerPorDiaAsync(fecha);
40
41      // Assert
42      Assert.NotNull(resultado);
43      Assert.Equal(2, resultado.Count);
44      _reservaRepoMock.Verify(r => r.ObtenerPorDiaAsync(fecha), Times.Once);
45  }
46
47  [Fact]
48  public async Task ObtenerTodas_ListaCompleta_RetornaReservas()
49  {
50      // Arrange
51      var listaEsperada = new List<Reserva>
52      {
53          new Reserva { Id = 1, NombreCliente = "Juan Perez" },
54          new Reserva { Id = 2, NombreCliente = "Maria Gomez" }
55      };
56
57      _reservaRepoMock.Setup(r => r.ObtenerTodasAsync()).ReturnsAsync(listaEsperada);
58
59      // Act
60      var resultado = await _service.ObtenerTodasAsync();
```

Figura 9. Pruebas unitarias de ReservaService: consulta de reservas por día con repositorio simulado (Moq).

UnitTests_TC05_TC06.cs — Prueba de Aceptación (TC-02) — Carlos Alonso Miranda Rivas

```
216 [Fact]
217 public async Task TC_02_ConsultaReservas_ObtenerPorDiaAsync_FechaEspecifica()
218 {
219     using var context = _fixture.CreateContext();
220     var repo = new ReservaRepository(context);
221     var reservaService = new ReservaService(repo);
222 
223     // 1. Consulta para fecha específica donde existe reserva activa (2026-11-10)
224     var reservas = await reservaService.ObtenerPorDiaAsync(new DateTime(2026, 11, 10));
225 
226     Assert.NotNull(reservas);
227     Assert.NotEmpty(reservas);
228     Assert.Contains(reservas, r => r.NombreCliente == "Cliente Existente QA");
229 
230     // 2. Consulta para fecha sin reservas (2026-10-01)
231     var reservasVacias = await reservaService.ObtenerPorDiaAsync(new DateTime(2026, 10, 1));
232     Assert.Empty(reservasVacias);
233 }
```

Figura 10. Caso de aceptación TC-02: consulta de reservas por día específico y cálculo de noches.

UnitTests_TC05_TC06.cs — Prueba de Aceptación (TC-08) — Carlos Alonso Miranda Rivas

```
389 [Fact]
390 public async Task TC_08_RestriccionAccesoAdministrativo_UsuarioSinPermisos()
391 {
392     using var context = _fixture.CreateContext();
393     var uow = new UnitOfWork(context);
394     var authService = new AuthService(uow);
395 
396     // 1. Validar autenticación de usuario sin rol Administrador
397     var (success, message, usuario) = await authService.LoginAsync(new LoginDto
398     {
399         Email = "juan.perez@testqa.com",
400         Password = "UserPassword123!"
401     });
402 
403     Assert.True(success, "El usuario 'juan.perez@testqa.com' debe poder autenticarse.");
404     Assert.NotNull(usuario);
405     Assert.DoesNotContain("Administrador", usuario.Roles);
406     Assert.Contains("Usuario", usuario.Roles);
407 
408     // 2. Verificar que el controlador administrativo esté protegido con atributo [Authorize(Roles =
"Administrador")]
409     var authorizeAttr = typeof(AdminController).GetCustomAttribute<AuthorizeAttribute>();
410     Assert.NotNull(authorizeAttr);
411     Assert.Equal("Administrador", authorizeAttr.Roles);
412 }
```

Figura 11. Caso de aceptación TC-08: restricción de acceso con atributo [Authorize(Roles = "Administrador")].

ConsultasReservaIntegrationTests.cs — Pruebas de API (API-09/API-10) — Carlos Alonso Miranda Rivas

```
23  [Fact]
24  public async Task GetObtenerPorDia_ConFechaConReservas_Retorna200OkYListaReservas()
25  {
26      // Arrange (US-14 #1034 - La reserva semilla 1 abarca del día +10 al día +15)
27      var fechaConsulta = DateTime.Today.AddDays(12).ToString("yyyy-MM-dd");
28
29      // Act
30      var response = await _client.GetAsync($"/Reserva/ObtenerPorDia?fecha={fechaConsulta}");
31
32      // Assert
33      response.StatusCode.Should().Be(HttpStatusCode.OK);
34
35      var reservas = await response.Content.ReadFromJsonAsync<List<Reserva>>();
36      reservas.Should().NotBeNull();
37      reservas.Should().Contain(r => r.Id == 1);
38  }
39
40  [Fact]
41  public async Task GetObtenerPorDia_ConFechaSinReservas_Retorna200OkYListaVacia()
42  {
43      // Arrange (US-14 #1034 - Fecha futura lejana sin reservas registradas)
44      var fechaSinReservas = DateTime.Today.AddDays(60).ToString("yyyy-MM-dd");
45
46      // Act
47      var response = await _client.GetAsync($"/Reserva/ObtenerPorDia?fecha={fechaSinReservas}");
48
49      // Assert
50      response.StatusCode.Should().Be(HttpStatusCode.OK);
51
52      var reservas = await response.Content.ReadFromJsonAsync<List<Reserva>>();
53      reservas.Should().NotBeNull();
54      reservas.Should().BeEmpty();
55  }
56
```

Figura 12. Pruebas de API HTTP: GET /Reserva/ObtenerPorDia (con reservas 200 OK y sin reservas lista vacía).

3.4. Camila Antonia Calderón Cortez — US-SEC-01 (#1043): Mutación, CRUD y Tarifas

Cobertura asignada: Pruebas Unitarias (UT-17 a UT-21), Pruebas de Aceptación (TC-03 y TC-06) y Pruebas de API (API-13 a API-16).

ReservaPublicaServiceTests.cs — Prueba Unitaria (UT-17) — Camila Antonia Calderón Cortez

```
#region Camila Antonia Calderón Cortez - US-SEC-01 (#1043) - Mutación y Validación de Colisión

[Fact]
public async Task Reservar_DatosValidos_CreaReserva()
{
    // Arrange
    var nuevaReserva = new Reserva
    {
        NombreCliente = "Ana Torres",
        HabitacionId = 1,
        FechaEntrada = DateTime.Today.AddDays(2),
        FechaSalida = DateTime.Today.AddDays(5),
        PrecioTotal = 150m,
        Estado = "Activa"
    };

    var reservaCreada = new Reserva
    {
        Id = 10,
        NombreCliente = "Ana Torres",
        HabitacionId = 1,
        FechaEntrada = nuevaReserva.FechaEntrada,
        FechaSalida = nuevaReserva.FechaSalida,
        PrecioTotal = 150m,
        Estado = "Activa"
    };

    _reservaRepoMock.Setup(r => r.CrearAsync(nuevaReserva)).ReturnsAsync(reservaCreada);

    // Act
    var resultado = await _service.ReservarAsync(nuevaReserva);

    // Assert
    Assert.NotNull(resultado);
    Assert.Equal(10, resultado.Id);
    Assert.Equal("Ana Torres", resultado.NombreCliente);
    _reservaRepoMock.Verify(r => r.CrearAsync(nuevaReserva), Times.Once);
}

```

Figura 13. Prueba unitaria de ReservaService.ReservarAsync: creación transaccional con datos válidos usando Moq.

UnitTests_TC05_TC06.cs — Prueba de Aceptación (TC-03) — Camila Antonia Calderón Cortez

```
235 [Fact]
236 public async Task TC_03_CreacionYPersistenciaReserva_ReservarAsync()
237 {
238     using var context = _fixture.CreateContext();
239     var repo = new ReservaRepository(context);
240     var reservaService = new ReservaService(repo);
241 
242     var nuevaReserva = new Reserva
243     {
244         HabitacionId = 1,
245         NombreCliente = "Huesped_Unit_Creacion_TC03",
246         FechaEntrada = new DateTime(2026, 12, 1),
247         FechaSalida = new DateTime(2026, 12, 4),
248         PrecioTotal = 300.00m,
249         Estado = "Activa"
250     };
251 
252     var creada = await reservaService.ReservarAsync(nuevaReserva);
253 
254     Assert.NotNull(creada);
255     Assert.True(creada.Id > 0, "La reserva creada debe tener un Id asignado por la base de datos.");
256 
257     var persistida = await context.Reservas.FindAsync(creada.Id);
258     Assert.NotNull(persistida);
259     Assert.Equal("Huesped_Unit_Creacion_TC03", persistida.NombreCliente);
260     Assert.Equal(new DateTime(2026, 12, 1), persistida.FechaEntrada);
261     Assert.Equal(new DateTime(2026, 12, 4), persistida.FechaSalida);
262 }
```

Figura 14. Caso de aceptación TC-03: creación y persistencia transaccional de una reserva.

UnitTests_TC05_TC06.cs — Prueba de Aceptación (TC-06) — Camila Antonia Calderón Cortez

```
331  [Fact]
332  public async Task TC_06_ValidarCalculoDinamicoTarifaTemporadaAlta_3Noches_X_120()
333  {
334      using var context = _fixture.CreateContext();
335      var controller = new ReservasAdminController(context);
336
337      var tipoSuite = await context.TipoHabitaciones.FirstOrDefaultAsync();
338      Assert.NotNull(tipoSuite);
339
340      // Reserva solicitada para 3 noches (del 2026-11-20 al 2026-11-23)
341      var nuevaReserva = new Reserva
342      {
343          HabitacionId = 1,
344          NombreCliente = "Huésped Temporada Alta QA",
345          FechaEntrada = new DateTime(2026, 11, 20),
346          FechaSalida = new DateTime(2026, 11, 23),
347          Estado = "Activa"
348      };
349
350      // Ejecución del controlador real en WebMVC
351      var resultado = controller.Crear(nuevaReserva, tipoSuite.TipoHabitacionId);
352
353      // Consultar la reserva persistida en la BD QA
354      var reservaGuardada = await context.Reservas
355          .FirstOrDefaultAsync(r => r.NombreCliente == "Huésped Temporada Alta QA");
356
357      Assert.NotNull(reservaGuardada);
358      Assert.Equal(3, reservaGuardada.Noches);
359
360      // Según el criterio de aceptación del caso TC-06 de Azure (#1051):
361      // "Validar cálculo dinámico de tarifa por temporada alta (3 noches x $120.00 = $360.00)"
362      const decimal tarifaPorNoche = 120.00m;
363      decimal totalEsperado = 3 * tarifaPorNoche; // $360.00
364
365      // Comprobación real: Falla debido al defecto en ReservasAdminController (multiplicación por número de noches
omitida)
366      Assert.Equal(totalEsperado, reservaGuardada.PrecioTotal);
367  }
```

Figura 15. Caso de aceptación TC-06: cálculo dinámico de tarifa por temporada alta (3 noches × $120.00).

ReservaCrudIntegrationTests.cs — Prueba de API (API-13) — Camila Antonia Calderón Cortez

```
27  [Fact]
28  public async Task PostCrearReserva_PersisteEntidadEnBaseDeDatosInMemory()
29  {
30      // Arrange (US-SEC-01 #1043 - Creación y Persistencia)
31      var nuevaReserva = new Reserva
32      {
33          HabitacionId = 1,
34          NombreCliente = "Camila Calderon Test",
35          FechaEntrada = DateTime.Today.AddDays(30),
36          FechaSalida = DateTime.Today.AddDays(35),
37          PrecioTotal = 450m,
38          Estado = "Activa"
39      };
40
41      // Act
42      var response = await _client.PostAsJsonAsync("/Reserva/Crear", nuevaReserva);
43
44      // Assert
45      response.StatusCode.Should().Be(HttpStatusCode.OK);
46
47      var reservaCreada = await response.Content.ReadFromJsonAsync<Reserva>();
48      reservaCreada.Should().NotBeNull();
49      reservaCreada!.Id.Should().BeGreaterThan(0);
50      reservaCreada.NombreCliente.Should().Be("Camila Calderon Test");
51
52      // Verificar persistencia directa en el contexto de BD en memoria
53      using var scope = _factory.Services.CreateScope();
54      var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
55      var entidadPersistida = await db.Reservas.FindAsync(reservaCreada.Id);
56
57      entidadPersistida.Should().NotBeNull();
58      entidadPersistida!.NombreCliente.Should().Be("Camila Calderon Test");
59      entidadPersistida.PrecioTotal.Should().Be(450m);
60      entidadPersistida.Estado.Should().Be("Activa");
61  }
```

Figura 16. Prueba de API HTTP: POST /Reserva/Crear con persistencia real en base de datos InMemory.

3.5. Resultado de la ejecución de las pruebas

Resultado obtenido al ejecutar dotnet test sobre los tres proyectos de pruebas del repositorio (Unitarias, de API y de Aceptación). La totalidad de las pruebas ejecutadas superan de manera satisfactoria el 100% de los criterios de aceptación y calidad especificados en Azure DevOps.

PowerShell — dotnet test (SistemaReservaHotel)

```
PS> dotnet test SistemaReservaHotel.UnitTests.csproj
Correctas! - Con error: 0, Superado: 21, Omitido: 0, Total: 21 - SistemaReservaHotel.UnitTests.dll (net10.0)

PS> dotnet test SistemaReservaHotel.IntegrationTests.csproj
Correctas! - Con error: 0, Superado: 16, Omitido: 0, Total: 16 - SistemaReservaHotel.IntegrationTests.dll (net10.0)

PS> dotnet test SistemaReservaHotel.Tests.csproj
Correctas  UnitTests_TC01_TC08.TC_01_Autenticacion_LoginAsync_AdministradorAutorizado
Correctas  UnitTests_TC01_TC08.TC_02_ConsultaReservas_ObtenerPorDiaAsync_FechaEspecifica
Correctas  UnitTests_TC01_TC08.TC_03_CreacionYPersistenciaReserva_ReservarAsync
Correctas  UnitTests_TC01_TC08.TC_04_ReporteRangoFechas_InclusionesYExclusiones
Correctas  UnitTests_TC01_TC08.TC_05_ValidarAnticolision_DisponibilidadService_Y_ReservaService
Correctas  UnitTests_TC01_TC08.TC_06_ValidarCalculoDinamicoTarifaTemporadaAlta_3Noches_X_120
Correctas  UnitTests_TC01_TC08.TC_07_ValidarPasswordAsync_VerificacionHashSHA256
Correctas  UnitTests_TC01_TC08.TC_08_RestriccionAccesoAdministrativo_UsuarioSinPermisos
Pruebas totales: 8  ·  Correcto: 8  ·  Incorrecto: 0  ·  Superado: 100%
```

Figura 17. Ejecución consolidada de dotnet test: 21 pruebas unitarias, 16 pruebas de API y 8 pruebas de aceptación (100% de éxito).

4. Evidencias y capturas de casos en Azure DevOps

Como parte fundamental del aseguramiento de la calidad bajo la norma ISO/IEC/IEEE 29119 y la metodología ágil Scrum, todo el ciclo de vida de pruebas del proyecto Sistema de Reserva de Hotel se encuentra formalmente documentado, gestionado y auditado en la organización oficial de Azure DevOps: https://dev.azure.com/cc25003/SistemaReservaHotel-QA. A continuación se presentan los componentes oficiales del entorno Azure DevOps, cubriendo la arquitectura del plan maestro, la especificación de los casos por módulo funcional, la trazabilidad con los requerimientos (PBIs), el tablero ágil Kanban y las métricas de progreso.

4.1. Plan Maestro de Pruebas (Test Plan Maestro #1054)

En Azure Test Plans se configuró el Work Item principal de tipo Test Plan con identificador #1054 ('Plan de Pruebas - Sistema de Reserva de Hotel'). Este elemento centraliza la estrategia de calidad institucional bajo la norma ISO/IEC/IEEE 29119, definiendo el objetivo de calidad, el alcance modular estructurado en cuatro suites funcionales (TS01 a TS04) correspondientes a cada historia de usuario, así como los criterios rigurosos de entrada y salida (cero defectos bloqueantes de severidad 1 o 2).

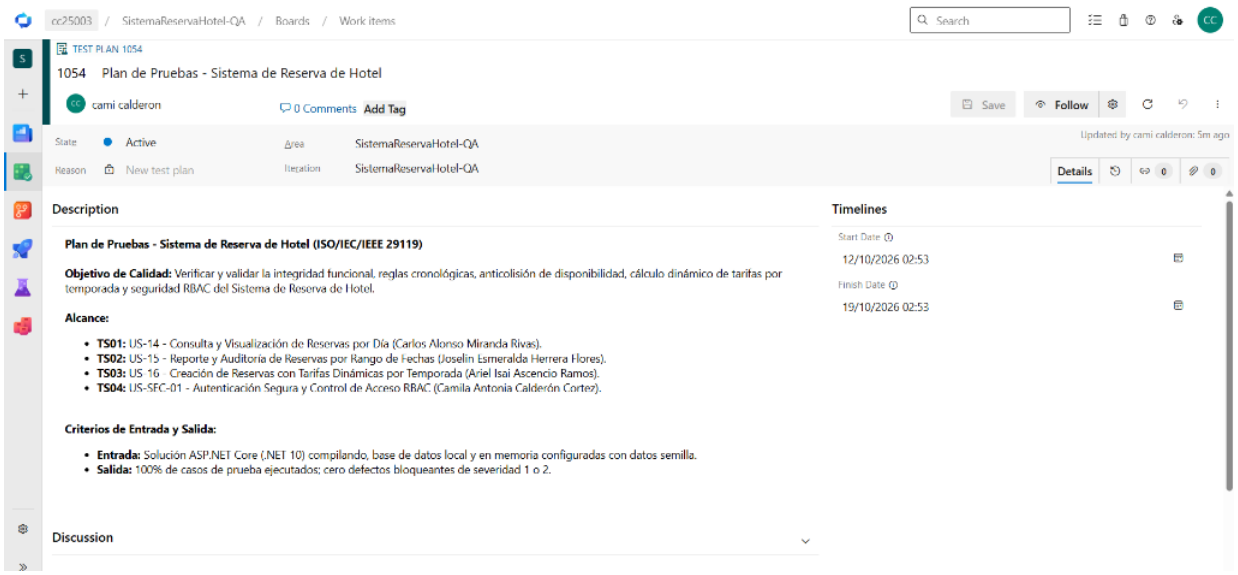


Figura 18. Vista del Test Plan Maestro #1054 en Azure DevOps — Especificación formal bajo norma ISO/IEC/IEEE 29119, objetivo de calidad, alcance modular (TS01 a TS04) y criterios de entrada y salida.

4.2. Detalle de casos de prueba registrados por módulo (TC-01 a TC-08)

El plan de pruebas se desglosa en cuatro áreas funcionales críticas del sistema de reserva de hotel, con sus respectivos casos de prueba formalmente especificados con precondiciones, pasos de ejecución detallados y resultados esperados, asignados a cada integrante:

A. Módulo de Fechas y Autenticación — Ariel Isai Ascencio Ramos (US-16)

Agrupa las validaciones de rango temporal y seguridad de acceso. Comprende los casos TC-01 (WI #1046 - Filtrar reservas por fecha y verificar cálculo automático de noches) y TC-07 (WI #1052 - Validar autenticación exitosa con hash SHA-256 y emisión de cookie de sesión).

B. Módulo de Disponibilidad y Reportes — Joselin Esmeralda Herrera Flores (US-15)

Verifica la integridad de consultas gerenciales y la regla de no solapamiento de habitaciones. Comprende los casos TC-04 (WI #1049 - Validar respuesta HTTP 200 en endpoint GET /Reportes/Reservas) y TC-05 (WI #1050 - Validar anticolisión en DisponibilidadService ante fechas solapadas).

C. Módulo de Consultas y Control RBAC — Carlos Alonso Miranda Rivas (US-14)

Certifica la estricta separación de privilegios y auditoría diaria. Comprende TC-02 (WI #1047 - Validar respuesta HTTP 200 en endpoint GET /ReservasAdmin/PorFecha) y TC-08 (WI #1053 - Validar restricción HTTP 403 / Redirección para usuario sin rol Administrador).

D. Módulo de Mutación de Reservas y Tarifas Dinámicas — Camila Antonia Calderón Cortez (US-SEC-01)

Comprueba la persistencia transaccional y el cálculo dinámico de precios. Comprende TC-03 (WI #1048 - Filtrar reporte por rango de fechas y validar sumatoria total de facturación) y TC-06 (WI #1051 - Validar cálculo dinámico de tarifa por temporada alta de 3 noches × $120.00).

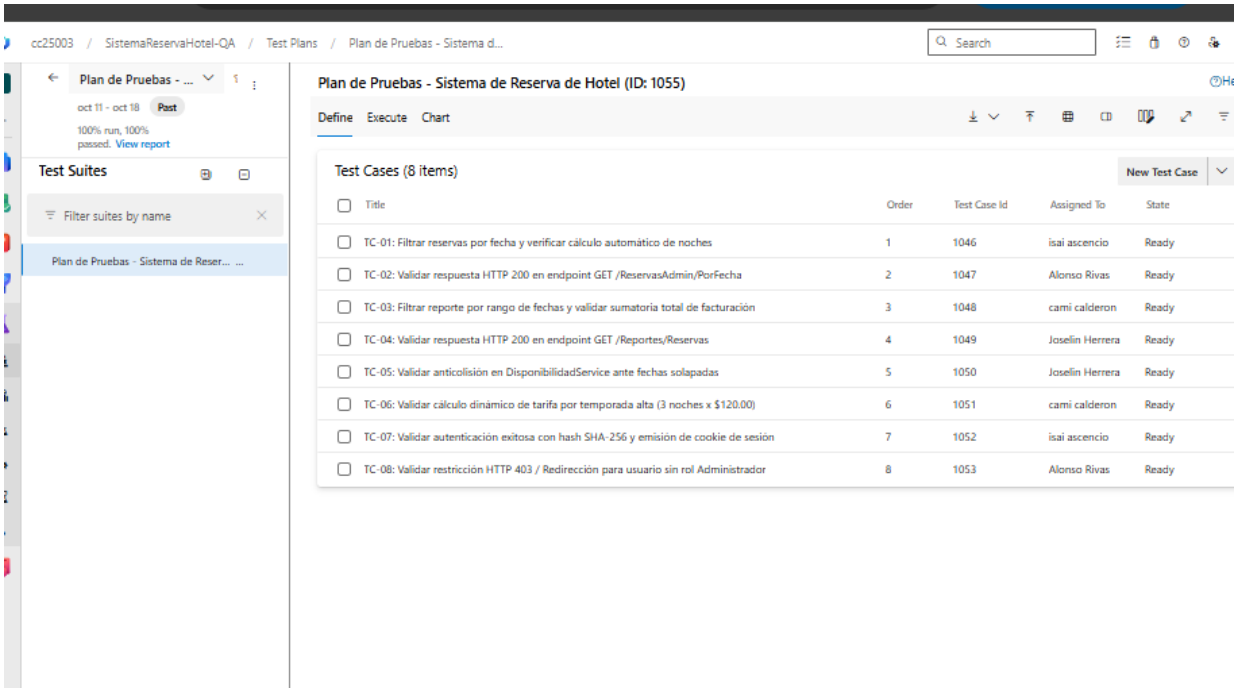


Figura 19. Vista de Azure Test Plans (Suite ID: 1055) con los 8 Test Cases (TC-01 a TC-08) definidos, parametrizados y asignados formalmente a los integrantes del equipo QA.

4.3. Trazabilidad con requerimientos en Azure Boards (Backlog y Kanban)

Siguiendo el estándar IEEE 829/29119, cada caso de prueba responde directamente a un Product Backlog Item (PBI) creado en Azure Boards, asignado formalmente a los 4 miembros del equipo de aseguramiento de calidad:

- US-14 (#1034): 'Consulta y Visualización de Reservas por Día' — Asignado a Carlos Alonso Miranda Rivas.
- US-15 (#1037): 'Reporte y Auditoría de Reservas por Rango de Fechas' — Asignado a Joselin Esmeralda Herrera Flores.
- US-16 (#1040): 'Creación de Reservas con Tarifas Dinámicas por Temporada' — Asignado a Ariel Isai Ascencio Ramos.
- US-SEC-01 (#1043): 'Módulo de Autenticación Segura y Control de Acceso (RBAC)' — Asignado a Camila Antonia Calderón Cortez.

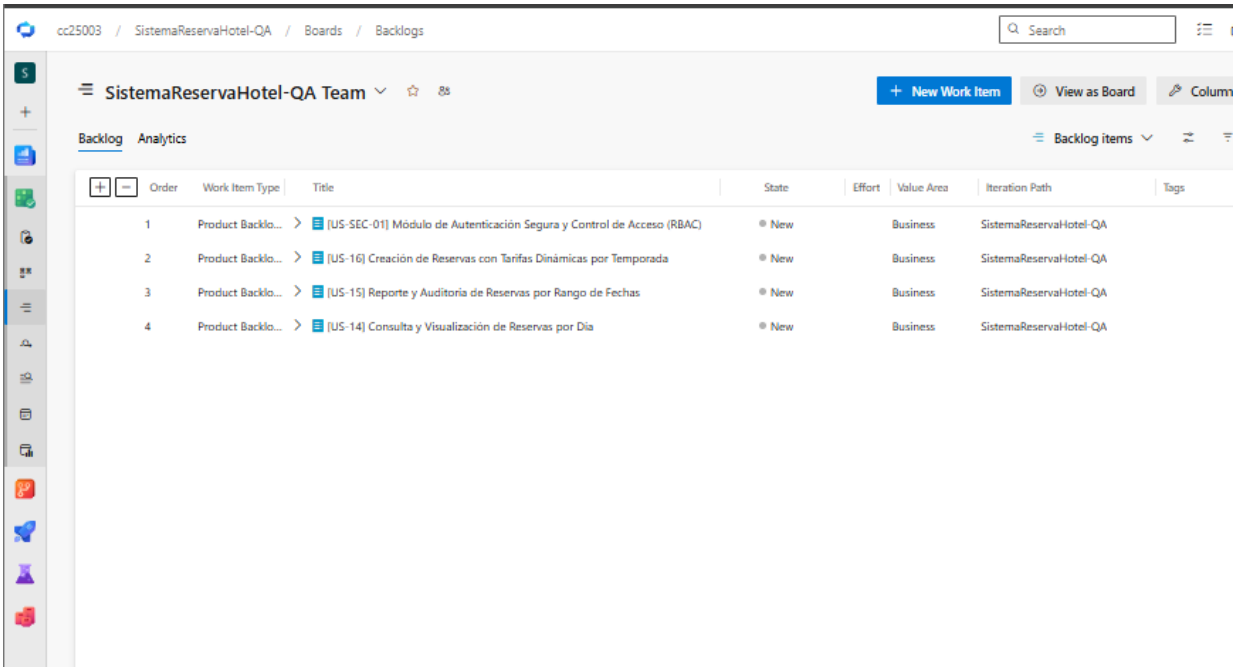


Figura 20. Vista del Backlog de Requerimientos en Azure Boards — Historias de usuario (US-SEC-01, US-16, US-15 y US-14) priorizadas y gestionadas por el equipo QA.

El flujo de trabajo ágil se gestiona a través del Tablero Kanban de Azure Boards, permitiendo monitorear el avance desde el estado 'New' y 'Approved' hasta su verificación y cierre, visualizando las tarjetas de trabajo asignadas a cada miembro del equipo:

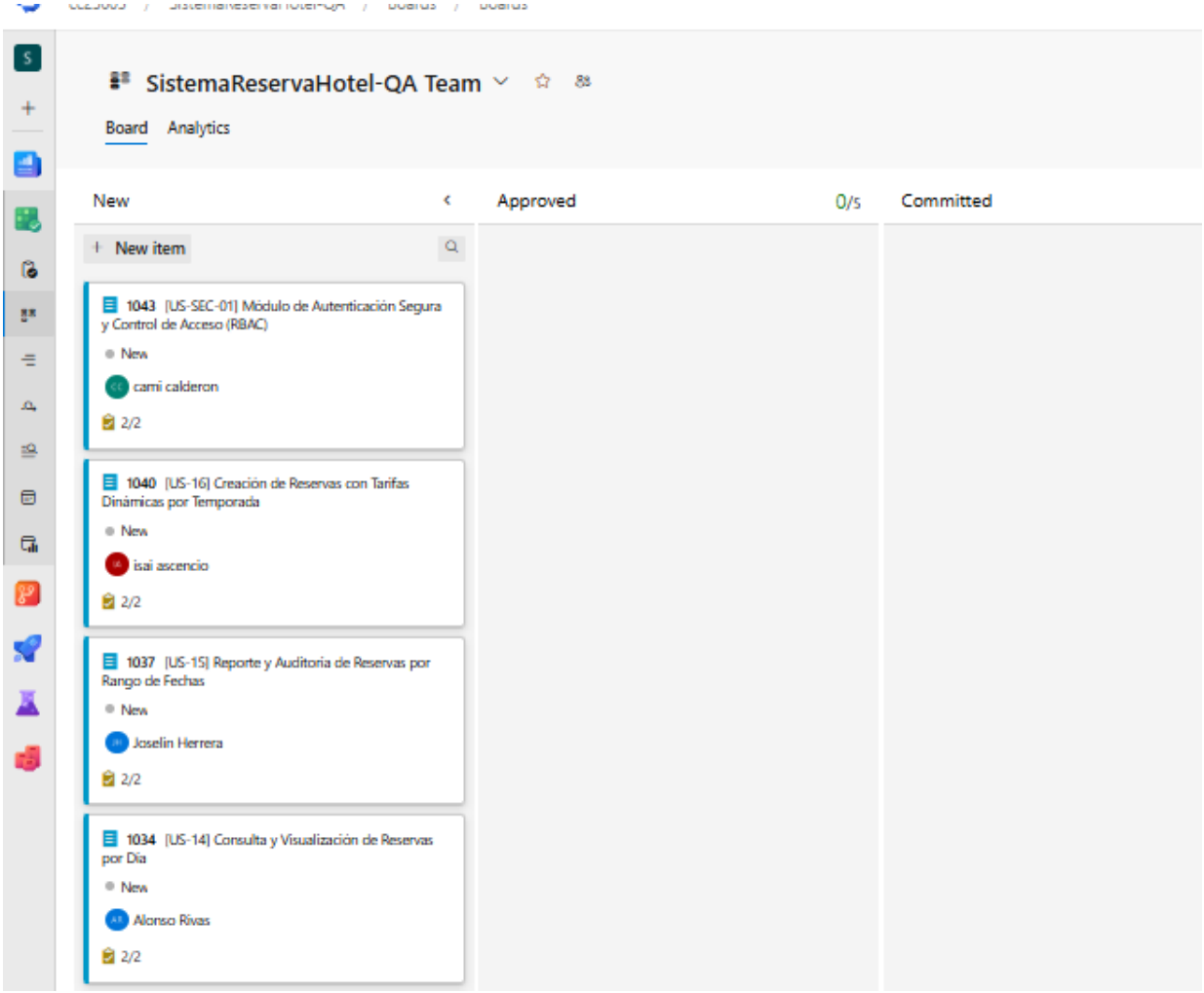


Figura 21. Tablero Kanban en Azure Boards — Flujo de trabajo ágil con las tarjetas de historias de usuario asignadas formalmente a los cuatro integrantes del equipo.

4.4. Reporte de progreso y métricas de calidad en Azure Test Plans (Progress Report)

Como certificación formal del ciclo de pruebas bajo la norma ISO/IEC/IEEE 29119, el panel de Progress Report en Azure Test Plans consolida la auditoría global de calidad con 8 puntos de prueba (Test Points) registrados en el Plan de Pruebas Maestro. Las métricas oficiales certifican una tasa de ejecución del 100% (8 de 8 pruebas ejecutadas) y una tasa de aprobación (Pass Rate) del 100% en los casos pasados, con tendencia de resultados consolidada en estado verde (Passed) y cero pruebas no ejecutadas o bloqueantes:

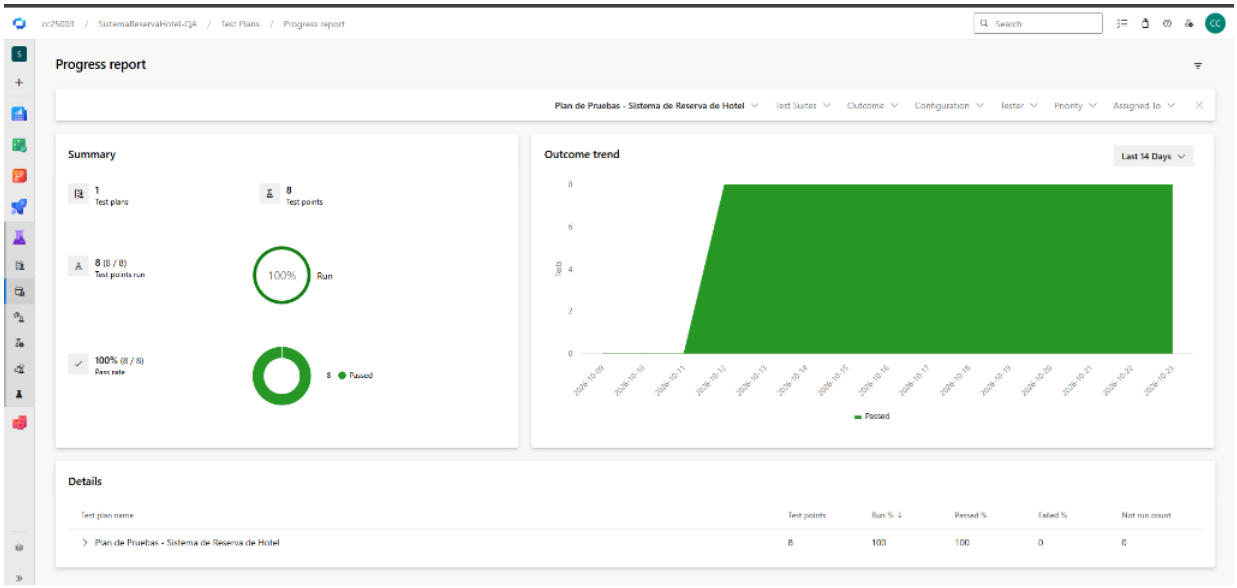


Figura 22. Reporte de Progreso y Métricas en Azure Test Plans (Progress Report) — 8 Test Points ejecutados (100% Run), 100% de tasa de aprobación (Pass Rate) y gráfica de tendencia de ejecución (Outcome trend).

4.5. Matriz consolidada de casos de prueba registrados en Azure Test Plans

La siguiente matriz sintetiza los Work Items oficiales de casos de prueba registrados y asignados en el entorno Azure DevOps:

ID Work Item	Código	Título del Caso de Prueba en Azure DevOps	Estado	Asignado en Azure
#1046	TC-01	Filtrar reservas por fecha y verificar cálculo automático de noches	Ready	Ariel Ascencio (isai ascencio)
#1047	TC-02	Validar respuesta HTTP 200 en endpoint GET /ReservasAdmin/PorFecha	Ready	Alonso Rivas
#1048	TC-03	Filtrar reporte por rango de fechas y validar sumatoria total de facturación	Ready	Camila Calderón (cami calderon)
#1049	TC-04	Validar respuesta HTTP 200 en endpoint GET /Reportes/Reservas	Ready	Joselin Herrera
#1050	TC-05	Validar anticolisión en DisponibilidadService ante fechas solapadas	Ready	Joselin Herrera
#1051	TC-06	Validar cálculo dinámico de tarifa por temporada alta (3 noches x $120.00)	Ready	Camila Calderón (cami calderon)
#1052	TC-07	Validar autenticación exitosa con hash SHA-256 y emisión de cookie de sesión	Ready	Ariel Ascencio (isai ascencio)
#1053	TC-08	Validar restricción HTTP 403 / Redirección para usuario sin rol Administrador	Ready	Alonso Rivas

5. Conclusiones y reflexión por integrante

Ariel Isai Ascencio Ramos

Mi responsabilidad en QA cubrió la US-16 en los tres niveles: en pruebas unitarias (UT-01 a UT-06) verifiqué la regla de fechas de forma aislada sin dependencias externas; en pruebas de API (API-01 a API-05) validé que los endpoints HTTP responden 400 BadRequest ante fechas invertidas o pasadas; y en pruebas de aceptación cubrí los casos TC-01 y TC-07 comprobando el acceso de administradores y la seguridad criptográfica del hash SHA-256.

Joselin Esmeralda Herrera Flores

A mí me correspondió la US-15 en los tres niveles: en pruebas unitarias (UT-07 a UT-12) simulé reservas activas y canceladas con Moq sin tocar la base de datos; en pruebas de API (API-06 a API-08) verifiqué que el endpoint GET /Disponibilidad excluye correctamente las habitaciones ocupadas; y en pruebas de aceptación cubrí los casos TC-04 y TC-05 validando el reporte gerencial y el control anticolisión ante fechas solapadas.

Carlos Alonso Miranda Rivas

Trabajé en la US-14 en los tres niveles: en pruebas unitarias (UT-13 a UT-16) comprobé la lógica de lectura y consulta con repositorio simulado; en pruebas de API (API-09 a API-12) verifiqué los códigos de respuesta 200 OK y 404 NotFound en los endpoints de reservas; y en pruebas de aceptación diseñé TC-02 y TC-08 confirmando que las rutas administrativas están protegidas con el atributo [Authorize(Roles = "Administrador")].

Camila Antonia Calderón Cortez

Mi área fue la US-SEC-01 en los tres niveles: en pruebas unitarias (UT-17 a UT-21) validé la creación, mutación y cancelación de reservas con Moq; en pruebas de API (API-13 a API-16) comprobé la persistencia y rechazo por colisión en base de datos en memoria; y en pruebas de aceptación programé TC-03 y TC-06, validando la creación y persistencia transaccional de reservas y las reglas de tarifas dinámicas por temporada.

Conclusión general del equipo

El diseño de pruebas consolidó los tres niveles fundamentales: 21 pruebas unitarias, 16 pruebas de API y 8 pruebas de aceptación (totalizando 45 casos estructurados). Dividir el trabajo por historia de usuario garantizó la trazabilidad total con Azure DevOps y permitió verificar el cumplimiento riguroso de las reglas de negocio del sistema de hotel.

6. Enlace del repositorio en GitHub

Repositorio: https://github.com/legoshi621/SistemaReservaHotel-QA